

Tworzenie agentów AI w .NET

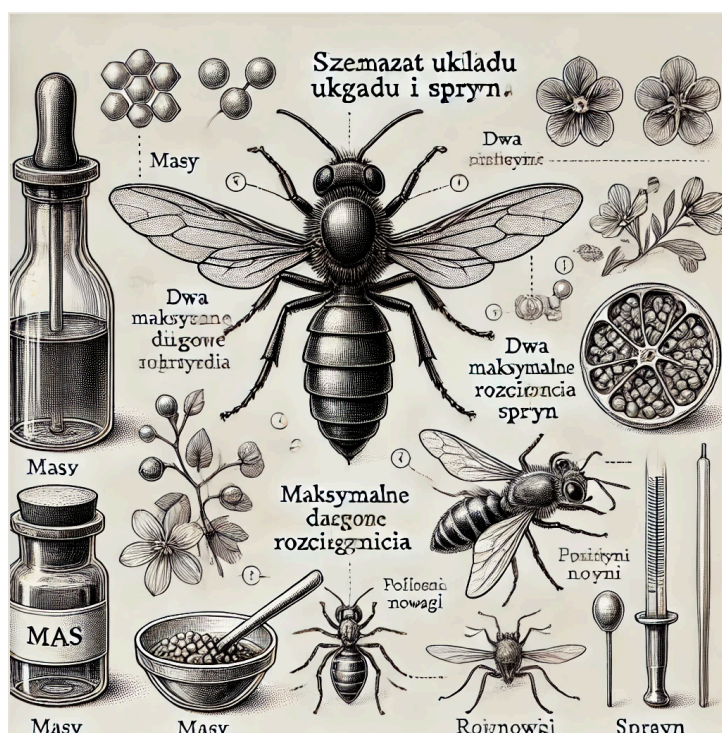
Marcin Sulecki

marcin.sulecki@sulmar.pl

Halucynacja

Skoro AI halucynuje i podaje błędne odpowiedzi...

...to czy ma w ogóle jakieś sensowne zastosowanie?



Przykład halucynacji modelu

Wiarygodna forma ≠ prawdziwa treść

Demo – TUP Lab

AI nie tylko odpowiada na pytania – może realizować cel i działać w rzeczywistym systemie.

Tworzenie agentów AI w .NET

Spróbujemy zrozumieć, jak projektować systemy, w których AI jest jednym z narzędzi.

Kim jest agent?

Agent otrzymuje cel, obserwuje sytuację, podejmuje decyzje i wykonuje działania.

Systemy agentowe istniały dużo wcześniej. Decyzja wcale nie musi pochodzić z LLM. Może być oparta na regułach, automacie stanów, algorytmie planowania, heurystyce czy klasycznym modelu ML (Machine Learning).

Agent AI to agent, który działa z wykorzystaniem LLM.

Autonomia – zdolność do działania bez ciągłego kierowania przez człowieka i bez programowania każdego kroku.

Cechy agenta:

- orientacja na cel
- planowanie i wnioskowanie
- pozyskiwanie informacji i działanie
- adaptacja

Agent w środowisku

Agent działa w pętli:

środowisko → obserwacja → decyzja → działanie → środowisko

W klasycznym agencie programista określa sposób podejmowania decyzji:

Jeżeli student jest w budynku Mini i chce dojść do sali 204, znajdź w grafie najkrótszą ścieżkę i prowadź według kolejnych węzłów.

Po wprowadzeniu LLM możemy **część decyzji** pozostawić modelowi:

Student: „Chyba się zgubiłem. Widzę automat z kawą i jakieś schody. Mam zajęcia u Suleckiego. Jak na nie trafić?”.

Płacimy za to wysoką cenę: **decyzje przestają być w pełni deterministyczne i przewidywalne.**

Historia LLM

Transformer → GPT → instruction following → ChatGPT → reasoning + tools stworzyły fundament współczesnych agentów.

- 2017: „Attention Is All You Need” → Transformer
- 2018: GPT → generative pre-training
- instruction following → model wykonuje polecenie, a nie tylko dokańcza tekst
- 2022: ChatGPT → technologia trafia do masowego użytkownika
- reasoning + tools → model wnioskuje i sięga po narzędzia

LLM vs Agent

LLM generuje odpowiedź. Agent wykorzystuje model, aby realizować cel i podejmować działania.

Agent $\neq$ LLM

Chatbot rozmawia. Agent realizuje cel.

Chatbot: „Sala 204 znajduje się na drugim piętrze”.

Agent: Chcesz dojść do sali 204. Sprawdzę, gdzie jesteś, znajdę salę, wyznaczę trasę i będę prowadził Cię krok po kroku.

Nowe narzędzie

Agent to nowe narzędzie w skrzynce inżyniera – nie powód, żeby wyrzucić pozostałe.

Dobry inżynier nie zastępuje wszystkiego LLM. Wyznacza granicę między tym, co powinno być probabilistyczne, a tym, co powinno pozostać deterministyczne.

Inżynier i beton

Nie pytamy: „Jak użyć tutaj AI?”

Pytamy: „Która część problemu rzeczywiście wymaga AI?”

TUP Żak – pojęcia

Na jednym przykładzie zobaczymy elementy, które przez cały semestr będziemy kolejno odkrywać i implementować.

- **Prompt & Context** – jak ograniczać nieporozumienia
- **Tools** – jak dać dostęp do rzeczywistych danych i działań
- **MCP** – jak te możliwości udostępniać
- **RAG** – jak oprzeć odpowiedź na wiedzy
- **State & Memory** – co zachować
- **Evaluation** – jak sprawdzić, czy działa
- **Security** – czego agentowi nie wolno zrobić samodzielnie

Następny wykład

Skoro agent ma realizować nasze cele, to jak powiedzieć mu, czego od niego oczekujemy?